ChatGPT planning prompt

How to use it:

1. Copy everything below the line "PROMPT STARTS HERE" into ChatGPT. Use a model with file creation turned on, so it can build the PDF. A plain-text copy is in `3-chatgpt-prompt.txt`.
2. If ChatGPT asks questions, answer them.
3. Download the PDF it creates and paste it into Claude Code. Claude Code then builds the project task by task.

PROMPT STARTS HERE (copy everything below)

You are the lead architect, data scientist and ML engineer for my project. Your job is planning only: produce the complete plan and framework. Another AI coding agent (Claude Code) will do the actual building by following your plan exactly, so the plan must be complete, concrete and unambiguous. Do not write the project code itself, except short snippets that define an interface, a schema or a config.

Your final output must be a downloadable PDF, structured as described in section H. Before writing it, ask me up to 10 short questions about anything you truly cannot decide yourself, such as my laptop's RAM and GPU, how many hours a week I have, or my current skill level. If I tell you to skip the questions, make reasonable assumptions and list them in the PDF.

A. About me and the goal

- I am a student in Kerala building a portfolio for data analyst, data scientist and ML engineer roles. I already have data and ML projects. One of them, kerala-veg-prices, uses 26.5 million Agmarknet wholesale price records (2021 to May 2026), Open-Meteo rain data and a 16-day price-spike warning model (PR-AUC 0.33 against a chance level of 0.04). This new project may reuse that data.
- The project is inspired by KW Live (kw-live.com), a private-beta "regional spatial intelligence" map of Waterloo Region, Canada. KW Live shows what is happening now. Our project must also predict what happens next and publish how accurate those predictions were.
- Goal: one serious portfolio project that shows data engineering, data analysis, data science, machine learning, deep learning and LLM work together, presented on a live public map.

B. Hard constraints (never break these)

1. **Total cost Rs 0.** Free tiers and open data only. No paid API, no paid hosting, no credit card. The Claude API is paid, so the Ask feature must use a free LLM tier (for example Gemini API free tier, Groq free tier or Cloudflare Workers AI) or a local model. For every free tier you rely on, name its limit and give a fallback.
2. **No reuse** of the KW Live name, logo, visual identity or website text. Propose 5 original project names with available-sounding domains.
3. **Build order:** Karnataka (state 1), then Kerala (state 2), then Delhi (state 3). Build a shared platform once with Karnataka and reuse it for the other two.
4. **Only real data.** Use the verified sources in section D. If you suggest any other source, mark it "UNVERIFIED" and add a task to test it before anything depends on it. Never invent URLs, dataset names or API fields.
5. **Honest ML.** Split data by time (no random splits for time series). Every model must beat a simple baseline (persistence, climatology or seasonal naive). Report error with confidence intervals. No leakage. No claims beyond what was tested.
6. **Environment:** Windows 10 laptop. Use a Python 3.12 virtual environment (Python 3.14 is installed, but some libraries lack wheels for it). Use DuckDB 1.x, not 2.0.0, which corrupted files in an earlier project. Heavy deep-learning training runs on Kaggle or Google Colab free GPUs.
7. **Executable by an AI agent.** Every task must be small enough for one Claude Code session (about 1 to 3 hours of work), with exact files to create, commands to run and a test that proves it is done.

C. Facts already established (do not redo this research)

- No Indian state has open live feeds for road closures, police or fire calls, traffic cameras or radio scanners. Use the substitutes listed in section E.
- Karnataka has the most open data, Kerala is strongest for floods, dams and prices, and Delhi is closest to KW Live (live bus GPS).
- KW Live's features, for reference: live dark map, radius filter, roads layer, live bus tracking, train routes, public-safety calls, scanner-derived reports, police history, cameras, flights, weather alerts, permits, building outlines, waste-day lookup, Starlink view, view modes (wall, atlas globe), time window, sound and read-aloud alerts, feed health badges, event ticker, newsroom panel, daily brief, market ticker, Place Lens (click a place to see now, property, around, history, watch, sources), source provenance labels, Ask by text and voice with citations, place watches, and City Memory (what was known at a past time).

D. Verified data sources (tested live on 4 November 2026)

Free with no key, working for all states:

- Open-Meteo forecast API (api.open-meteo.com/v1/forecast)
- Open-Meteo historical weather, ERA5 since 1940 (archive-api.open-meteo.com/v1/archive)
- Open-Meteo flood API, GloFAS river discharge, past and forecast (flood-api.open-meteo.com/v1/flood). No river cell for Lakshadweep, Puducherry or the Andaman Islands.
- Open-Meteo air quality, a model estimate (air-quality-api.open-meteo.com)
- Open-Meteo ensemble API, 40 members (ensemble-api.open-meteo.com)
- Open-Meteo climate API, CMIP6 projections to 2050 (climate-api.open-meteo.com)
- NASA POWER daily point data (power.larc.nasa.gov)
- IMD Pune 0.25-degree daily gridded rainfall, 1901 to 2024, in NetCDF (imdpune.gov.in; Python library imdlib)
- NDMA Sachet CAP alert RSS: sachet.ndma.gov.in/cap_public_website/rss/rss_<state>.xml (works for 28 of 36 states, for example rss_kerala.xml and rss_karnataka.xml) plus rss_india.xml for all states
- adsb.lol (api.adsb.lol/v2/point/{lat}/{lon}/{nm}) and OpenSky for live flights. Free, but rate-limited after about 15 fast calls, so cache results.
- CelesTrak satellite orbit data (celestrak.org)
- NASA GIBS WMTS satellite tiles, for example the epsg3857 MODIS_Terra_CorrectedReflectance_TrueColor layer
- AWS Terrain Tiles, Terrarium elevation PNGs (s3.amazonaws.com/elevation-tiles-prod)
- OpenFreeMap vector basemap (tiles.openfreemap.org)
- WorldPop 100 m population rasters for India (hub.worldpop.org)
- geoBoundaries India ADM2, 736 districts
- Geofabrik OSM extracts for India's zones (the Overpass API refused queries, so use the extracts)
- Google Open Buildings (India covered)
- USGS earthquake API
- datameet railways on GitHub (stations.json, trains.json, schedules.json; the data is old)
- India Data Portal (ISB) agricultural marketing data (Agmarknet). The Agmarknet site itself is a JavaScript app.
- Telegram Bot API

Free, but needs a free key or registration: data.gov.in API (mandi prices, CPCB station AQI), NASA FIRMS fire hotspots (MAP_KEY), aisstream.io ships, Delhi Open Transit Data (live GTFS-realtime bus positions).

Karnataka: KSNDMC telemetric rain gauges (ksndmc.org), KWRIS rainfall analytics and reservoirs (water.karnataka.gov.in), OpenCity Bengaluru datasets including KSNDMC hobli rainfall 2017 to 2023 (data.opencity.in), BMTC GTFS and Namma Metro GTFS (Mobility Database), Karnataka RERA (rera.karnataka.gov.in; needs scraping).

Kerala: KSEB daily reservoir levels for 16 dams (kseb.in/dashboard/view/waterlevel), KSEB Dam Safety monthly statistics (dams.kseb.in; file links load with JavaScript, so a browser scrape is needed and the number of years is unknown), KSDMA dam level PDFs (sdma.kerala.gov.in/dam-water-level), Kerala Floods 2018 dataset (Kaggle), the

KSDMA post-disaster needs assessment report, the 2018 landslide inventory (research literature), and the ISRO Landslide Atlas.

Delhi: Open Transit Data (otd.delhi.gov.in; static and real-time GTFS after registration), CPCB stations through data.gov.in.

Other states, for reference only: Chennai lake levels monthly since 1945 (cmwssb.tn.gov.in), Telangana TGDPS weather stations, Andhra Pradesh APSDPS weather stations, Gujarat reservoirs, Maharashtra WRD dams, and a Himachal landslide inventory (Mendeley).

Failed or blocked in testing (plan around them): India-WRIS (SSL certificate error from Python; test in a browser), Kochi Metro open data (timeout), Kerala RERA (HTTP 503), NCRB (connection failed), CGWB groundwater (failed), ASDMA flood report page (404), Bihar FMISC (site gone), GDELT (HTTP 429 rate limit).

E. Features to plan (all of these, plus any you add)

Map and layers: live state map; radius filter; time slider and replay; 3D terrain and globe; wall mode; feed health badges; event log and ticker; daily satellite image layer; satellites overhead; stations plus scheduled transit positions (live in Delhi); fire hotspots and Sachet alerts including lightning; news-derived event pins labelled preliminary (our scanner substitute); flood-exposed roads (our road-closure substitute); RERA projects; building outlines; mandi price ticker; district crime totals if a source exists.

Place Lens: now, risk, forecast, history, nearby and sources for any ward, taluk or address; official, model or inferred labels on every item; nearest hospital, shelter, police and fire station; shareable place report.

ML layers: dam and reservoir forecasts; ward flood risk; landslide risk; drought; AQI; bus delay; price spikes; a "what if" rain simulator; SHAP explanations; uncertainty bands; a model accuracy page with daily backtests; an anomaly feed.

People and climate: population at risk; safe route to a shelter that avoids flood-risk roads; heatwave layer; crop health (NDVI); rainfall trend since 1901; CMIP6 outlook to 2050; monsoon tracker; past-disaster replay.

Ask, alerts and memory: RAG Ask by text and voice with citations; Kannada, Malayalam, Hindi and English; place watches; Telegram bot and web push; daily brief; City Memory (every snapshot stored, "as known then").

Data and access: analysis dashboards; data quality dashboard; open data download and public API; offline PWA; embeddable widgets; state comparison.

Add more features if they make the project stronger and still cost Rs 0. Mark each feature MVP, v1 or stretch.

F. ML work already proposed (expand and decide)

- **Karnataka:** Bengaluru ward flood risk; reservoir level and inflow forecast at 7 and 30 days; rain-gauge gap filling and spatial interpolation (kriging against ML); taluk drought forecast (SPI); Kolar tomato price-spike warning using rain; analysis of BMTC coverage gaps.
- **Kerala:** dam level forecast at 7 days; district or taluk flood-risk classifier tested on 2018; landslide susceptibility map; rain-to-vegetable-price effect (event study and Granger causality); stretch goals of Sentinel-1 flood mapping with U-Net and Malayalam and English news NLP; transfer test of the Karnataka flood model on Kerala.
- **Delhi:** bus delay and ETA prediction; bus bunching anomaly detection; station AQI forecast for 24 to 72 hours that must beat the CAMS model; heatwave risk map; pollution exposure along bus routes.
- **Cross-state:** RAG Ask with an evaluation set, anomaly detection, a model monitoring and drift dashboard, and state comparison.

G. What the plan must decide and explain (miss nothing)

1. **Architecture:** system diagram; data flow from source to raw, clean, features, models, predictions and the website; the folder and repository structure as a full file tree; module boundaries; config files; naming conventions.
2. **Tools, with exact versions and the reason for each:** languages, libraries (data, geospatial, ML, deep learning, NLP, LLM, web map, frontend), storage formats, orchestration (GitHub Actions schedules), data versioning (for

example DVC), experiment tracking (for example MLflow locally), testing (pytest, data tests such as pandera or Great Expectations), linting and formatting, CI, documentation, frontend framework, map library, charting, hosting, and secrets handling for the free keys. For each choice, name one rejected alternative and why.

3. **Data engineering, for each source:** fetch method, schedule, rate limits, retries, parsing (HTML, PDF, JSON, NetCDF, GTFS, GTFS-realtime), schema with column names and types, units, time zone (Asia/Kolkata), spatial reference (EPSG codes), quality checks, deduplication, gap handling, raw-data retention, snapshotting for City Memory, storage size estimate against the free-tier limits, and its licence and attribution text.
4. **Spatial layer:** the place identity model (ward, taluk, district, address and building IDs); joins between datasets; H3 hexagons versus admin areas (decide); geocoding with free tools only; map tile strategy; vector tiles (PMTiles) for large layers.
5. **Data analysis:** a list of EDA notebooks with the questions each answers, every chart to produce, and the dashboards with their exact metrics.
6. **Data science:** hypotheses to test, statistical tests (with assumptions and corrections for multiple tests), causal-inference methods for the rain-to-price question, spatial statistics (Moran's I, kriging), and anomaly detection methods.
7. **For every ML model, a full model card plan:** problem type; target and its exact definition; prediction horizon; unit of prediction; label source; feature list with each feature's source and lag (proving no leakage); train, validation and test periods with dates; cross-validation scheme (rolling origin, spatial block); baseline(s); candidate algorithms, from simple to complex (for example linear, gradient boosting such as LightGBM or XGBoost, random forest, Prophet, ARIMA/SARIMAX, LSTM, GRU, Temporal Fusion Transformer, N-BEATS, graph neural networks for spatial dependence, U-Net for segmentation, transformer NLP models such as IndicBERT or MuRIL); the final pick and why; the hyperparameter search space and method; metrics with target thresholds (MAE, RMSE, MAPE, CRPS for probabilistic forecasts, PR-AUC, ROC-AUC, F1, Brier score, calibration); class-imbalance handling; uncertainty method (quantile regression, conformal prediction, ensembles); explainability (SHAP); failure modes; how it is retrained; and how it is monitored for drift.
8. **Deep-learning details** for each neural network you choose: architecture diagram in words (layers, sizes, activations), input tensor shapes, loss function, optimiser, learning-rate schedule, regularisation, early stopping, batch size, epochs, GPU memory estimate, training time estimate on free Kaggle or Colab GPUs, and export format (ONNX if useful).
9. **LLM and RAG:** document sources, chunking, embedding model (free, local), vector store (free), retrieval method, prompt templates, a citation format, guardrails (no emergency advice; always point to official sources), an evaluation set of at least 50 questions with expected answers, evaluation metrics (faithfulness, answer correctness, citation accuracy), the free LLM tier and its limits, and a fallback when the quota runs out.
10. **Frontend and product:** every screen, panel and component; the Place Lens layout; map interactions; the alert UX; accessibility (WCAG 2.2 AA); mobile layout; Indian-language support; offline PWA behaviour; and performance budgets.
11. **MLOps:** daily prediction job, model registry, model versioning, backtest job, drift detection, alerting when a feed or model breaks, and rollback.
12. **Testing:** unit, data-contract, integration, model-regression and end-to-end tests, with what each covers.
13. **Security and legal:** secrets, scraping etiquette (robots.txt, rate limits), licences and attribution for every source, privacy (no personal data), and disclaimers ("not an emergency service, not official").
14. **Project management:** phases, milestones and a week-by-week timeline for each state; the critical path; a risk register (risk, likelihood, impact, mitigation); and the definition of done for each phase.
15. **Portfolio output:** README structure, a case-study write-up outline for each model, demo video script, resume bullet points with placeholders for the real numbers, and interview talking points.
16. **ML concepts explained:** a short glossary that explains every ML, statistics and neural-network concept used in the plan, in simple English, so I can learn and defend them in interviews.

H. Output format (the PDF)

Produce one PDF with these sections in order:

1. Summary (one page)
2. Assumptions and answers to your questions
3. Decisions log (table: ID, decision, alternatives considered, reason)
4. Architecture and file tree
5. Tools and versions (table)
6. Data source catalogue (table: ID, source, URL, access, schedule, schema link, licence, status VERIFIED or UNVERIFIED)
7. Data schemas (one table per dataset)
8. Feature catalogue (table: ID, feature, state(s), MVP/v1/stretch, depends on)
9. Data analysis plan
10. Data science plan
11. ML model cards (one subsection per model, covering everything in G7 and G8)
12. LLM and RAG plan
13. Frontend plan
14. MLOps, testing, security and legal
15. Timeline, milestones and risk register
16. **Task list for Claude Code:** numbered tasks T001, T002 and so on, in build order. Each task has: ID, title, phase, state, depends on, files to create or change, step-by-step instructions, commands to run, acceptance test (exact command and expected result) and estimated hours. Group the tasks by phase. The first task must set up the repository; the second the Python 3.12 environment, linting and tests.
17. Portfolio and interview pack
18. Glossary of concepts
19. **Appendix A:** the full task list again as one YAML block (fields: id, title, phase, state, depends_on, files, steps, commands, acceptance, hours), so Claude Code can parse it.
20. **Appendix B:** open questions and items marked UNVERIFIED.

Write in clear, simple English. Use tables wherever possible. Be specific: name exact libraries, versions, file paths, column names, dates and numbers. Do not leave anything as "TBD" without a task that decides it. If the PDF would be too long for one response, produce it in parts, and tell me which part comes next.